EX17 - Modules and Imports - Splitting the Project into Files

Beginner Step-by-Step Guide

This guide follows the real file `ex17_vertical_motion_modules.zip`. It explains the problem first and then the code from the perspective of someone starting Python.

1. What we want to solve

Split the vertical-motion simulation into vertical_motion.py, data_processing.py and main.py so each file has a clear responsibility.

Problem variables / elements

Name	Meaning
vertical_motion.py	physics, validation, simulation and reference
data_processing.py	trajectory post-processing
main.py	configuration and orchestration

Equation or logic on paper

```
main.py -> functions imported from vertical_motion.py and data_processing.py
```

Manual check

Physical results should not change from EX16. The change is architectural: each responsibility now lives in a different module.

2. What we already know and what is new

Already seen:

- functions
- dictionaries
- NumPy
- masks

New in this exercise:

- module = .py file
- from ... import ...
- per-module dependencies
- def main()
- __name__
- main guard
- separation of responsibilities

3. New Python concepts

1. Module

Syntax: `vertical_motion.py`

A module is a Python file that can define functions, classes or constants reusable from other files.

2. Internal import

Syntax: `from vertical_motion import validate_inputs`

main.py imports a specific function and can then call it directly.

3. __name__

Syntax: `__name__`

Python assigns this special name to each module. When a file is executed directly it equals "__main__".

4. Main guard

Syntax: `if __name__ == "__main__": main()`

Runs main() only when this file is launched directly, not when it is imported.

4. Code step by step

Step 1. vertical_motion.py

```
def validate_inputs(config): ...
def simulate_vertical_motion(config): ...
def euler_step(...): ...
def analytical_solution(config): ...
```

This file contains the motion model and related calculations. It also imports sqrt and NumPy because its functions use them.

Step 2. data_processing.py

```
def process_trajectory(trajectory_array, config):
    ...
    return processed_results, processed_data
```

This module does not control the flight: it receives an already-computed trajectory_array and filters, summarizes and augments it with energy.

Step 3. Import functions into main.py

```
from vertical_motion import validate_inputs, simulate_vertical_motion, analytical_solution, calculate_errors
from data_processing import process_trajectory
```

main.py connects modules without copying their implementations.

Step 4. Define main()

```
def main():
    config = {
        "h0": 0.0,
        "v0": 50.0,
        "t0": 0.0,
        "g": 9.81,
        "dt": 0.1
    }
```

main() is the orchestrator: it prepares inputs and decides the call order.

Step 5. Flow between modules

```
results, trajectory_array = simulate_vertical_motion(config)
processed_results, processed_data = process_trajectory(trajectory_array, config)
```

Data returned by one function is passed to another. Modules communicate through parameters and return values.

Step 6. Protect main

```
if __name__ == "__main__":
    main()
```

When main.py is run directly the condition is True. If main.py were imported elsewhere, the workflow would not start automatically.

5. Execution flow

```
main.py
 -> validate_inputs (vertical_motion)
 -> simulate_vertical_motion
 -> analytical_solution / calculate_errors
 -> process_trajectory (data_processing)
 -> print results
```

6. Expected result / check

Running main.py from the exercise folder produces the same simulation/post-processing results as before, now with separated responsibilities.

7. Common beginner mistakes

- Forgetting each module must import its own dependencies.
- Putting top-level execution code that runs on import.
- Creating circular imports.
- Confusing importing a function with calling it.

8. If you remember only five things

- A module is a .py file.
- import connects files.
- main.py orchestrates.
- Data travels through parameters and returns.
- The main guard prevents accidental execution on import.

9. Mini-practice

Change one input. Before running, write a comment predicting what should change and why. Then run the file and compare your prediction with the output.

Completion criterion

Move on when you can explain the problem, equation/logic, new Python concepts and execution flow without simply reading the code.